

Simulations pour le screening sortie ZIP avec variables supplémentaires V13_parallélisation - Version commentée

J. Dedecker, M.-L. Taupin et A.-S. Tocquet

2026-06-29

Contents

1	Etapes des calculs:	2
1.1	Simulation du design	2
1.2	Calcul du prédicteur linéaire	3
1.3	Génération des Y_i	3
1.4	2) Calcul des $\hat{\tau}^{(j)}$	3
1.5	3) Calcul des $\hat{\tau}^{(j)}$ pour les variables non liées aux Y_i	3
1.6	4) Calcul des intercept ($\hat{\tau}_0^{(j)}$) individuels pour les régression linéaires simples (variables par variables) en utilisant la formule:	3
1.7	6) Calcul de la variance de White en utilisant la formule:	4
1.8	7) On calcule ensuite les $\hat{\tau}^{(j)}$ renormalisés avec la formule:	4
1.9	8) On sait que pour les $j > 10$	4
1.10	9) Etape de Monte Carlo pour calculer les faux positifs et les faux négatifs sur N échantillons.	4
2	Implémentation R avec <code>screening_test_matrixR()</code> et	4
3	<code>reciprocal_screening_scoreR()</code>	4
3.1	Packages et fonctions de screening	4
3.2	Paramètres de simulation	5
3.3	Fonction de simulation du design	6
3.4	Simulation d'un jeu de données complet	7
3.5	Vérification ponctuelle de la statistique réciproque	8
3.6	Une réplication Monte Carlo	9
3.7	Monte Carlo avec parallélisation	10
3.8	Résultats	11
3.9	Sauvegarde	13

1 Etapes des calculs:

1.1 Simulation du design

On a 4 groupes de variables:

- Le groupe $\mathcal{A}$ qui contient les variables X_1 à X_7 à X_{10} et qui génèrent les Y_i
- Le groupe $\mathcal{B}$ qui contient des variables X_{11} à X_{14} qui sont des variables bruitées de X_1 , X_3 , X_4 et X_5 . Elles sont corrélées à X_1 , X_3 , X_4 et X_5 (qui génèrent les Y_i), mais elles ne génèrent pas les Y_i .
- Le groupe $\mathcal{C}$ qui contient des variables corrélées à X_{11} , X_{12} , X_{13} et X_{14} mais qui sont indépendantes des Y_i . Ces variables X_{15} , X_{16} , X_{17} et X_{18} ne doivent pas sortir: ce seront des faux positifs.
- Le groupe $\mathcal{D}$ contient les 1 – 18 variables indépendantes de toutes les autres variables du design et qui ne génèrent pas les Y_i

Plus précisément, le design est tel que:

- X_0 est l'intercept.
- X_1, X_2, X_3, X_4, X_5 sont des gaussiennes dépendantes qui interviennent toutes dans la génération des Y_i , comme l'intercept. Leur matrice de covariance est la matrice de Toeplitz suivante (avec $\rho = 0.57$ et $\sigma = 1$)

$$\Sigma = \begin{pmatrix} 1 & \rho & \rho^2 & \rho^3 & \rho^4 \\ \rho & 1 & \rho & \rho^2 & \rho^3 \\ \rho^2 & \rho & 1 & \rho & \rho^2 \\ \rho^3 & \rho^2 & \rho & 1 & \rho \\ \rho^4 & \rho^3 & \rho^2 & \rho & 1 \end{pmatrix}$$

Ensuite:

- $X_6 \sim \mathcal{B}(0.35)$ indépendantes des autres variables du design. La variable X_6 intervient dans la génération des Y_i
- $X_7 \sim \chi^2(2)$. La variable X_7 intervient dans la génération des Y_i .
- $X_8 = X_1 * \mathcal{P}(2)$. La variable X_8 intervient dans la génération des Y_i , mais la loi de Poisson n'intervient pas seule dans la génération des Y_i .
- $X_9 = X_2 * \mathcal{N}(1, 1)$ les deux étant indépendantes. La variable X_9 intervient dans la génération des Y_i , mais pas la $\mathcal{N}(1, 1)$ seule.
- $X_{10} = \mathcal{S}\sqcup(14)$. La variable X_{10} intervient dans la génération des Y_i .

1.1.0.1 Les autres variables:

- $X_{11} = X_1 + \mathcal{N}(0, 04^2) = X_1 + Z_3$.
- $X_{12} = X_3 + \mathcal{N}(0, 02^2) = X_3 + Z_4$.
- $X_{13} = X_4 + \mathcal{N}(0, 0.35^2) = X_4 + Z_5$.
- $X_{14} = X_5 + \mathcal{N}(0, 055^2) = X_5 + Z_6$.

- $X_{15} = Z_3$
- $X_{16} = Z_4$
- $X_{17} = Z_5$
- $X_{18} = Z_6$

The variables $X^{(19)}, X^{(20)}, \dots, X^{(1000)}$ are independent of $(Y, X^{(1)}, \dots, X^{(18)})$. These variables form a stationary Gaussian sequence with mean 0 and variance 1, and covariance matrix similar to the matrix Σ above (but now it is a 986×986 square matrix), with $\rho = 0.7$.

1.2 Calcul du prédicteur linéaire

$$X\theta^* = \theta_0^* + \theta_1^* X^{(1)} + \dots + \theta_{10}^* X^{(10)}.$$

1.3 Génération des Y_i

1.3.0.1 The output follows a Poisson distribution with an excess of 0: The output Y is generated as follows:

- Set θ^* such that

$$\theta^* = (-1, 3.619350, -3.274923, 2.963273, -2.681280, 2, 4, 6, 3, 2, 4)^T.$$

- $Y = Y_1 * Y_2$;
- $Y_1 \sim \mathcal{B}(\pi(X^{(1)}, \dots, X^{(10)}))$, with

$$P(Y_1 = 1 | X^{(1)}, \dots, X^{(10)}) = \frac{1}{1 + \exp(-\theta_0^* - \theta_1^* X^{(1)} - \dots - \theta_{10}^* X^{(10)})},$$

- $Y_2 \sim \mathcal{P}(|X\theta^* = \theta_0^* + \theta_1^* X^{(1)} + \dots + \theta_{10}^* X^{(10)}|)$.

Remarque: $\theta^* = \alpha$ où α_j sont les valeurs dans l'article.

1.4 2) Calcul des $\hat{\tau}^{(j)}$

On rappelle que pour $j = 1, \dots, p$,

$$\hat{\tau}^{(j)} = \frac{\text{Cov}_n(\mathbf{X}^{(j)}, \mathbf{Y})}{\text{Var}_n(\mathbf{X}^{(j)})}.$$

1.5 3) Calcul des $\hat{\tau}^{(j)}$ pour les variables non liées aux Y_i

1.6 4) Calcul des intercept ($\hat{\tau}_0^{(j)}$) individuels pour les régression linéaires simples (variables par variables) en utilisant la formule:

$$\hat{b}^{(j)} = \bar{Y} - \hat{\tau}^{(j)} \bar{X}^{(j)}.$$

5) Calcul des résidus $\hat{\varepsilon}_i^{(j)}$

Puis on définit les résidus liés aux régressions linéaires simples (variable par variable) en utilisant que pour $j = 1, \dots, 1000$,

$$\hat{\varepsilon}_i^{(j)} = Y_i - \hat{b}^{(j)} - \hat{\tau}^{(j)} X_i^{(j)}.$$

1.7 6) Calcul de la variance de White en utilisant la formule:

$$\hat{V}^{(j)} = \frac{n^{-1} \sum_{i=1}^n \left[(X_i^{(j)} - \bar{\mathbf{X}}^{(j)})^2 (\hat{\varepsilon}_i^{(j)})^2 \right]}{\text{Var}_n^2(\mathbf{X}^{(j)})}.$$

1.8 7) On calcule ensuite les $\hat{\tau}^{(j)}$ renormalisés avec la formule:

$$\hat{\gamma}^{(j)} = \frac{\sqrt{n} \hat{\tau}^{(j)}}{\sqrt{\hat{V}^{(j)}}},$$

où

$$\hat{V}^{(j)} = \frac{n^{-1} \sum_{i=1}^n \left[(X_i^{(j)} - \bar{\mathbf{X}}^{(j)})^2 (\hat{\varepsilon}_i^{(j)})^2 \right]}{\text{Var}_n^2(\mathbf{X}^{(j)})}.$$

1.9 8) On sait que pour les $j > 10$

$$\hat{\gamma}^{(j)} \xrightarrow[n \rightarrow +\infty]{\mathcal{L}} \mathcal{N}(0, 1).$$

A noter que pour $j = 1, \dots, 10$ il faut recentrer et écrire:

$$\left(\hat{\gamma}^{(j)} - \gamma^{(j)} \right) \xrightarrow[n \rightarrow +\infty]{\mathcal{L}} \mathcal{N}(0, 1),$$

où

$$\left(\hat{\gamma}^{(j)} - \gamma^{(j)} \right) = \frac{\sqrt{n} \left(\hat{\tau}^{(j)} - \tau^{(j)} \right)}{\sqrt{\hat{V}^{(j)}}},$$

avec

$$\tau^{(j)} = \frac{\text{Cov}(Y, X^{(j)})}{\text{Var}(X^{(j)})}, \quad \text{et} \quad V^{(j)} = \frac{\mathbb{E} \left[\left(X^{(j)} - \mathbb{E}(X^{(j)}) \right)^2 \varepsilon^{(j)2} \right]}{\left(\text{Var}(X^{(j)}) \right)^2}.$$

1.10 9) Etape de Monte Carlo pour calculer les faux positifs et les faux négatifs sur N échantillons.

2 Implémentation R avec `screening_test_matrixR()` et

3 `reciprocal_screening_scoreR()`

Cette version ne calcule plus manuellement les pentes, les résidus, les variances de White ou les statistiques de screening. Chaque réplication appelle `screening_test_matrixR()`. En mode "reciprocal", cette fonction doit appeler votre fonction `reciprocal_screening_scoreR()` en interne.

3.1 Packages et fonctions de screening

```
library(parallel)
library(knitr)
```

3.2 Paramètres de simulation

Le test est choisi une seule fois par le paramètre `screening_test` :

- "studentized" : sélection si $|T_j| \geq \Phi^{-1}(1 - q_n/2)$;
- "reciprocal" : sélection si $D_j^2 \leq 1/\{\Phi^{-1}(1 - q_n/2)\}^2$.

```
n <- params$n
N <- params$N
Dtotal <- params$Dtotal
q_n <- params$q_n
screening_test <- match.arg(
  params$screening_test,
  choices = c("studentized", "reciprocal")
)

# Nombre de variables : 10 variables dans X, puis 8 variables supplémentaires
# (X11,...,X18), puis les variables gaussiennes de bruit.
D_signal <- 10L
D_proxy_noise <- 8L
Dtilde <- Dtotal - D_signal - D_proxy_noise

if (Dtilde < 1L) {
  stop("Dtotal doit être au moins égal à 19.", call. = FALSE)
}

# Variables actives ou corrélées à des variables actives, comme dans le script
# d'origine. Les variables 15:Dtotal sont utilisées pour le taux de faux positifs.
signal_index <- 1:14
null_index <- 15:Dtotal

# Paramètres du design
d <- 5L
mu <- rep(0, d)
sigma <- 1
rho <- params$rho
mu_tilde <- rep(0, Dtilde)
rho_tilde <- params$rho_tilde
sigma_tilde <- 1

# Intercept puis dix coefficients.
theta_star <- c(
  -1,
  3.619350, -3.274923, 2.963273, -2.681280, 2,
  4, 6, 3, 2, 4
)

stopifnot(length(theta_star) == D_signal + 1L)
```

```

gamma_threshold <- qnorm(1 - q_n / 2)
distance_threshold <- 1 / gamma_threshold^2

parameter_table <- data.frame(
  parameter = c(
    "n", "N", "Dtotal", "Dtilde", "q_n", "test",
    "seuil studentisé", "seuil réciproque"
  ),
  value = c(
    n, N, Dtotal, Dtilde, q_n, screening_test,
    gamma_threshold, distance_threshold
  )
)

kable(parameter_table, caption = "Paramètres de simulation")

```

Table 1: Paramètres de simulation

parameter	value
n	1500
N	500
Dtotal	2000
Dtilde	1982
q_n	0.15
test	reciprocal
seuil studentisé	1.43953147093846
seuil réciproque	0.482567058515325

3.3 Fonction de simulation du design

```

simu_design <- function(n, d, mu, rho, sigma = 1) {
  if (length(mu) != d) {
    stop("mu doit avoir une longueur égale à d.", call. = FALSE)
  }
  if (!is.numeric(rho) || length(rho) != 1L || abs(rho) >= 1) {
    stop("rho doit appartenir à (-1, 1).", call. = FALSE)
  }

  # Matrice de covariance de Toeplitz. Elle est définie positive pour |rho| < 1.
  Sigma <- toeplitz(rho^(0:(d - 1L))) * sigma^2

  design <- MASS::mvrnorm(n = n, mu = mu, Sigma = Sigma)
  if (d == 1L) {
    design <- matrix(design, ncol = 1L)
  }

  colnames(design) <- paste0("V", seq_len(d))

  list(
    design = design,

```

```

    Sigma = Sigma
  )
}

```

3.4 Simulation d'un jeu de données complet

```

simu_one_dataset <- function(
  n,
  Dtotal,
  d,
  mu,
  rho,
  sigma,
  Dtilde,
  mu_tilde,
  rho_tilde,
  sigma_tilde,
  theta_star
) {
  # X1,...,X5 : gaussiennes corrélées
  design_mat <- simu_design(
    n = n,
    d = d,
    mu = mu,
    rho = rho,
    sigma = sigma
  )$design

  # Bruits des variables X11,...,X18.
  # Les deux arguments mean = 0 et sd = ... sont explicitement indiqués.
  Z3 <- rnorm(n, mean = 0, sd = 0.40)
  Z4 <- rnorm(n, mean = 0, sd = 0.02)
  Z5 <- rnorm(n, mean = 0, sd = 0.35)
  Z6 <- rnorm(n, mean = 0, sd = 0.55)

  # X1,...,X10
  X_signal <- cbind(
    design_mat,
    X6_bern = rbinom(n, size = 1, prob = 0.35),
    X7_chisq2 = rchisq(n, df = 2),
    X8_X1_Pois2 = design_mat[, 1] * rpois(n, lambda = 2),
    X9_X2_Norm = design_mat[, 2] * rnorm(n, mean = 1, sd = 1),
    X10_t14 = rt(n, df = 14)
  )

  colnames(X_signal)[1:5] <- paste0("X", 1:5)

  # Génération de Y : modèle ZIP défini dans le document.
  linear_predictor <- as.numeric(theta_star[1] + X_signal %*% theta_star[-1])
  y1 <- rbinom(n, size = 1, prob = plogis(linear_predictor))
  y2 <- rpois(n, lambda = abs(linear_predictor))
  y <- as.numeric(y1 * y2)
}

```

```

# X11,...,X18 et les variables indépendantes de bruit.
X_extra <- cbind(
  X11_X1_Z3 = design_mat[, 1] + Z3,
  X12_X3_Z4 = design_mat[, 3] + Z4,
  X13_X4_Z5 = design_mat[, 4] + Z5,
  X14_X5_Z6 = design_mat[, 5] + Z6,
  X15_Z3 = Z3,
  X16_Z4 = Z4,
  X17_Z5 = Z5,
  X18_Z6 = Z6,
  simu_design(
    n = n,
    d = Dtilde,
    mu = mu_tilde,
    rho = rho_tilde,
    sigma = sigma_tilde
  )$design
)

colnames(X_extra)[9:ncol(X_extra)] <- paste0(
  "X", 19:Dtotal
)

X_complete <- cbind(X_signal, X_extra)

if (ncol(X_complete) != Dtotal) {
  stop("La matrice X_complete n'a pas Dtotal colonnes.", call. = FALSE)
}

list(
  X = X_complete,
  y = y
)
}

```

3.5 Vérification ponctuelle de la statistique réciproque

Cette vérification appelle directement `reciprocal_screening_scoreR()` pour X_1 . Le calcul Monte Carlo emploie ensuite `screening_test_matrixR()` sur toutes les colonnes.

```

example_data <- simu_one_dataset(
  n = n,
  Dtotal = Dtotal,
  d = d,
  mu = mu,
  rho = rho,
  sigma = sigma,
  Dtilde = Dtilde,
  mu_tilde = mu_tilde,
  rho_tilde = rho_tilde,
  sigma_tilde = sigma_tilde,
  theta_star = theta_star
)

```



```

)

example_reciprocal <- reciprocal_screening_scoreR(
  x = example_data$X[, "X1"],
  y = example_data$y
)

reciprocal_diagnostic <- data.frame(
  variable = "X1",
  A = example_reciprocal$A,
  D2 = example_reciprocal$D2,
  D = example_reciprocal$D,
  T = example_reciprocal$T
)

kable(
  reciprocal_diagnostic,
  caption = "Diagnostic direct de reciprocal_screening_scoreR() pour X1"
)

```

Table 2: Diagnostic direct de reciprocal_screening_scoreR() pour X1

variable	A	D2	D	T
X1	268.4479	0.0028553	0.0534348	18.71438

3.6 Une réplique Monte Carlo

La sélection est toujours obtenue avec `screening_test_matrixR()`. Ainsi :

- avec `screening_test = "studentized"`, la règle utilise la statistique studentisée;
- avec `screening_test = "reciprocal"`, la règle utilise D_j^2 , calculée à partir de `reciprocal_screening_scoreR()` par votre fonction matricielle.

```

run_one_replication <- function() {
  simulated_data <- simu_one_dataset(
    n = n,
    Dtotal = Dtotal,
    d = d,
    mu = mu,
    rho = rho,
    sigma = sigma,
    Dtilde = Dtilde,
    mu_tilde = mu_tilde,
    rho_tilde = rho_tilde,
    sigma_tilde = sigma_tilde,
    theta_star = theta_star
  )

  screening_fit <- screening_test_matrixR(
    X = simulated_data$X,

```

```

    y = simulated_data$y,
    q = q_n,
    test = screening_test
  )

  results <- screening_fit$results

  # La colonne `selected` est la seule utilisée pour calculer les performances.
  selected <- as.logical(results$selected)

  statistic <- if (screening_test == "studentized") {
    as.numeric(results$score)
  } else {
    as.numeric(results$D2)
  }

  if (length(selected) != Dtotal || length(statistic) != Dtotal) {
    stop("Résultat de screening incomplet.", call. = FALSE)
  }

  names(selected) <- rownames(results)
  names(statistic) <- rownames(results)

  list(
    selected = selected,
    statistic = statistic
  )
}

```

3.7 Monte Carlo avec parallélisation

Sous Windows, `mclapply()` n'est pas disponible : le script utilisera automatiquement `lapply()`.

```

available_cores <- parallel::detectCores(logical = FALSE)
if (is.na(available_cores) || available_cores < 1L) {
  available_cores <- 1L
}

n_cores <- min(as.integer(params$n_cores), available_cores)
n_cores <- max(1L, n_cores)

if (.Platform$OS.type == "windows" || n_cores == 1L) {
  repetitions <- lapply(seq_len(N), function(i) run_one_replication())
} else {
  repetitions <- parallel::mclapply(
    X = seq_len(N),
    FUN = function(i) run_one_replication(),
    mc.cores = n_cores,
    mc.set.seed = TRUE
  )
}

```

3.8 Résultats

```
selected_matrix <- do.call(
  rbind,
  lapply(repetitions, `[`, "selected")
)

statistic_matrix <- do.call(
  rbind,
  lapply(repetitions, `[`, "statistic")
)

variable_names <- names(repetitions[[1]]$selected)
colnames(selected_matrix) <- variable_names
colnames(statistic_matrix) <- variable_names

selection_rate <- colMeans(selected_matrix)
mean_selected_size <- mean(rowSums(selected_matrix))
false_positive_rate <- mean(selected_matrix[, null_index, drop = FALSE])
true_positive_rate <- mean(selection_rate[signal_index])

statistic_label <- if (screening_test == "studentized") {
  "T statistic"
} else {
  "D2 statistic"
}

first_18 <- seq_len(min(18L, Dtotal))

resultats_screening <- data.frame(
  variable = variable_names[first_18],
  taux_selection = round(selection_rate[first_18], 3),
  groupe = c(
    rep("signal / associé", 14L),
    rep("bruit corrélé", min(4L, length(first_18) - 14L))
  )
)

summary_screening <- data.frame(
  mesure = c(
    "Taux moyen de sélection des variables 1:14",
    "Taux de faux positifs (variables 15:Dtotal)",
    "Cardinal moyen de l'ensemble sélectionné",
    "Cardinal attendu sous sélection parfaite des 14 premières"
  ),
  valeur = c(
    true_positive_rate,
    false_positive_rate,
    mean_selected_size,
    length(signal_index) + q_n * length(null_index)
  )
)
```

```

kable(
  resultats_screening,
  caption = paste(
    "Taux de sélection des 18 premières variables - test",
    screening_test
  )
)

```

Table 3: Taux de sélection des 18 premières variables — test reciprocal

	variable	taux_selection	groupe
X1	X1	1.000	signal / associé
X2	X2	0.842	signal / associé
X3	X3	1.000	signal / associé
X4	X4	1.000	signal / associé
X5	X5	0.996	signal / associé
X6_bern	X6_bern	0.990	signal / associé
X7_chisq2	X7_chisq2	1.000	signal / associé
X8_X1_Pois2	X8_X1_Pois2	1.000	signal / associé
X9_X2_Norm	X9_X2_Norm	0.324	signal / associé
X10_t14	X10_t14	1.000	signal / associé
X11_X1_Z3	X11_X1_Z3	1.000	signal / associé
X12_X3_Z4	X12_X3_Z4	1.000	signal / associé
X13_X4_Z5	X13_X4_Z5	1.000	signal / associé
X14_X5_Z6	X14_X5_Z6	0.972	signal / associé
X15_Z3	X15_Z3	0.112	bruit corrélé
X16_Z4	X16_Z4	0.154	bruit corrélé
X17_Z5	X17_Z5	0.170	bruit corrélé
X18_Z6	X18_Z6	0.150	bruit corrélé

```

summary_screening_display <- summary_screening
summary_screening_display$ valeur <- round(summary_screening_display$ valeur, 3)

kable(
  summary_screening_display,
  caption = paste(
    "Résumé Monte Carlo - statistique stockée :",
    statistic_label
  )
)

```

Table 4: Résumé Monte Carlo — statistique stockée : D2 statistic

mesure	valeur
Taux moyen de sélection des variables 1:14	0.937
Taux de faux positifs (variables 15:Dtotal)	0.151
Cardinal moyen de l'ensemble sélectionné	313.976
Cardinal attendu sous sélection parfaite des 14 premières	311.900

3.9 Sauvegarde

```
simulation_output <- list(  
  parameters = list(  
    n = n,  
    N = N,  
    Dtotal = Dtotal,  
    q_n = q_n,  
    screening_test = screening_test,  
    gamma_threshold = gamma_threshold,  
    distance_threshold = distance_threshold  
  ),  
  selection_rate = selection_rate,  
  selected_matrix = selected_matrix,  
  statistic_matrix = statistic_matrix,  
  resultats_screening = resultats_screening,  
  summary_screening = summary_screening  
)  
  
if (isTRUE(params$save_results)) {  
  output_file <- sprintf(  
    "screening_%s_N%s_n%s_q%s.rds",  
    screening_test,  
    N,  
    n,  
    gsub("\\.", "", format(q_n, scientific = FALSE))  
  )  
  
  saveRDS(simulation_output, file = output_file)  
  cat("Résultats sauvegardés dans :", output_file, "\n")  
}
```

```
## Résultats sauvegardés dans : screening_reciprocal_N500_n1500_q015.rds
```